FPT UNIVERSITY

PROJECT REPORT

DESIGN TDM 4-Channel Multiplexer/Demultiplexer — Verilog RTL Project

Team: 3

Class: IC2010

Team members: CAO HỮU AN

PHẠM MINH TÙNG

MAI THẾ TÂM

LÊ THẾ TRƯỜNG

PHAN ANH MINH

TRẦN MINH TRIẾT

Lecturer: Dr. Phạm Thế Vinh

HO CHI MINH CITY, NOVEMBER 2026

MỤC LỤC

1. TỔNG QUAN HỆ THỐNG (SYSTEM OVERVIEW) ..... - 3 -
1.1 Giới thiệu đề tài ..... - 3 -
1.2 Nguyên lý hoạt động chính ..... - 3 -
2. SƠ ĐỒ KHỐI VÀ LUỒNG DỮ LIỆU (BLOCK DIAGRAM & DATA FLOW) ..... - 3 -
2.1 Sơ đồ khối hệ thống (Block Diagram) ..... - 3 -
3. CẤU TRÚC KHO LƯU TRỮ VÀ MÔ ĐUN (REPOSITORY & MODULE INDEX) ..... - 4 -
3.1 Cấu trúc thư mục dự án (Repository Structure) ..... - 4 -
3.2 Danh sách Module RTL (Module Index) ..... - 5 -
4. TIMING CONTRACT & GLUE LOGIC ..... - 5 -
4.1 Tín hiệu mux_select = (bit_count + 1)[4:3] (Logic nhìn trước - Look-Ahead) ..... - 5 -
4.2 Tín hiệu load và byte_boundary = (bit_count[2:0] == 3'b111) ..... - 6 -
4.3 Tín hiệu demux_select = channel_select (Trễ 1 xung nhịp) ..... - 6 -
4.4 Logic nạp trước khi Reset (Preload on Reset) ..... - 6 -
5. ĐẶC TẢ GIAO DIỆN CÁC MODULE (INTERFACE SPECIFICATIONS) ..... - 6 -
5.1 Module tdm_counter ..... - 6 -
5.2 Module tdm_mux (Multiplexer 4:1) ..... - 7 -
5.3 Module tx_shift_reg (Parallel-to-Serial Shift Register) ..... - 7 -
5.4 Module rx_shift_reg (Serial-to-Parallel Shift Register / SIPO) ..... - 8 -
5.5 Module tdm_demux (Demultiplexer / Byte Router) ..... - 8 -
5.6 Module tdm_top (Top-Level Integration) ..... - 9 -
6. XÁC THỰC VÀ KIỂM THỬ (VERIFICATION) ..... - 10 -
6.1 Kiểm thử hệ thống (python scripts/run_tests.py) ..... - 10 -
6.2 Đối chiếu với Mô hình chuẩn Python (Python Golden Model vs RTL) ..... - 10 -
7. KẾT QUẢ TỔNG HỢP VÀ PHÂN TÍCH VIVADO (SYNTHESIS & TIMING RESULTS) ..... - 10 -
7.1 Báo cáo tiêu thụ tài nguyên phần cứng (Resource Utilization) ..... - 10 -
7.2 Phân tích thời gian (Design Timing Summary @ 100 MHz / 10 ns) ..... - 11 -
8. PHÂN TÍCH SO SÁNH VÀ ĐÁNH ĐỔI THIẾT KẾ (COMPARISON & TRADE-OFF ANALYSIS) ..... - 11 -
8.1 So sánh Kiến trúc: Counter-Driven (Hiện tại) vs. FSM-Driven (Truyền thống) - 11 -
8.2 So sánh Giao tiếp: TDM Nối tiếp 1 đường dây vs. Truyền song song (Parallel Data Bus) ..... - 12 -
8.3 So sánh Phương pháp Kiểm thử: Python Golden Model vs. Verilog Testbench thuần ..... - 12 -

9. HẠN CHẾ VÀ HƯỚNG PHÁT TRIỂN (LIMITATIONS & FUTURE WORK) ..... - 13 -
9.1 Hạn chế hiện tại (Limitations) ..... - 13 -
9.2 Hướng phát triển tương lai (Future Work) ..... - 13 -
10. KẾT LUẬN (CONCLUSION) ..... - 13 -

1. TỔNG QUAN HỆ THỐNG (SYSTEM OVERVIEW)

1.1 Giới thiệu đề tài

Dự án thực hiện thiết kế một bộ ghép kênh (Multiplexer) và tách kênh (Demultiplexer) phân chia theo thời gian (Time Division Multiplexing - TDM) 4 kênh bằng ngôn ngữ Verilog RTL. Hệ thống cho phép truyền dữ liệu 8-bit từ 4 kênh riêng biệt (CH0 đến CH3 qua một đường truyền dữ liệu nối tiếp duy nhất (TDM_DATA) và khôi phục lại chính xác ở các đầu ra tương ứng (CH0_OUT đến CH3_OUT).

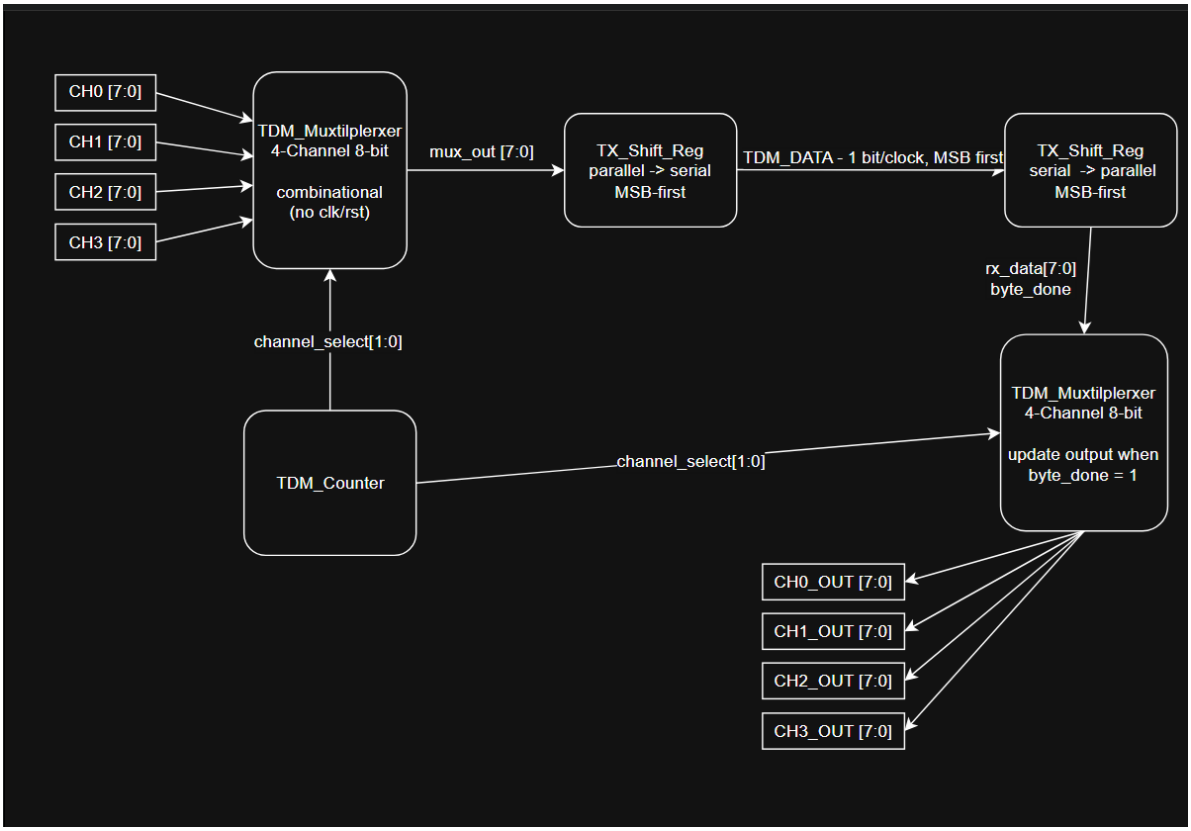
1.2 Nguyên lý hoạt động chính

- **Kiến trúc điều khiển bằng bộ đếm (Counter-driven / No FSM):** Hệ thống là một đường dữ liệu (datapath) hoàn toàn tuần hoàn, không sử dụng máy trạng thái hữu hạn (FSM), không có giao thức bắt tay (handshake) phức tạp hay cơ chế đồng bộ phụ.
- **Phân chia slot thời gian:**
    - Mỗi byte dữ liệu (8 bits) chiếm đúng 8 chu kỳ xung nhịp để truyền nối tiếp, ưu tiên bit có trọng số lớn nhất trước (MSB-first).
    - Một khung truyền (frame) gồm 4 byte liên tiếp theo thứ tự cố định: CH0 -> CH1 -> CH2 -> CH3 (tổng cộng 32 chu kỳ xung nhịp/khung).
- **Đồng bộ thời gian duy nhất:** Toàn bộ hệ thống thu/phát hoạt động dựa trên duy nhất 1 bộ đếm 5-bit chạy tự do (bit_count, đếm từ 0 -> 31 -> 0). Phía thu tự động nhận biết byte thu được thuộc về kênh nào để ghi vào thanh ghi đầu ra tương ứng mà không cần gửi kèm tín hiệu đồng bộ phụ.
- **Độ trễ hệ thống (Latency):**
    - **Khởi động lạnh (Cold start):** Sau khi reset, khung hợp lệ đầu tiên có sẵn sau **33 chu kỳ xung nhịp**.
    - **Thay đổi dữ liệu giữa dòng (Mid-stream change):** Mất từ **32 đến 40–41 chu kỳ xung nhịp** (trường hợp xấu nhất là 41 chu kỳ: 31 chu kỳ chờ slot nạp kênh + 8 chu kỳ truyền nối tiếp + 2 chu kỳ chốt đầu ra)

2. SƠ ĐỒ KHỐI VÀ LUỒNG DỮ LIỆU (BLOCK DIAGRAM & DATA FLOW)

2.1 Sơ đồ khối hệ thống (Block Diagram)

Dữ liệu được xử lý qua các khối chức năng chính: từ dữ liệu song song đầu vào, đi qua MUX, chuyển sang nối tiếp, truyền trên đường dây TDM_DATA, chuyển lại thành song song ở bên thu và đưa qua DEMUX ra các thanh ghi đầu ra.



3. CẤU TRÚC KHO LƯU TRỮ VÀ MÔ ĐUN (REPOSITORY & MODULE INDEX)

3.1 Cấu trúc thư mục dự án (Repository Structure)

- **src/ (Synthesizable RTL):**
  - tdm_counter.v: Bộ đếm 5-bit chạy tự do (nguồn thời gian chính).
  - tdm_mux.v: Bộ ghép kênh 4-to-1.
  - tx_shift_reg.v: Thanh ghi dịch song song sang nối tiếp.
  - rx_shift_reg.v: Thanh ghi dịch nối tiếp sang song song.
  - tdm_demux.v: Bộ định tuyến byte về các thanh ghi đầu ra CHX_OUT.
  - tdm_top.v: Module tích hợp top-level + lớp keo thời gian (glue layer).
- **techbench/:** Chứa testbench hệ thống (tb_tdm_top.v, tb_tdm_top_filebased.v) và tài liệu giải thích.
- **golden_model/:** Mô hình tham chiếu Python và các script so sánh tự động (run_verification.py, compare_output.py...).
- **input/:** Chứa các vector dữ liệu đầu vào (input_vectors.hex, random_vectors.hex).
- **scripts/:** Chứa script chạy testbench (run_tests.py) và tệp TCL tổng hợp Vivado (synth.tcl).
- **constraint/:** Tệp ràng buộc thời gian xung nhịp (constraints.xdc).

3.2 Danh sách Module RTL (Module Index)

STT	MODULE	TỆP NGUỒN	VAI TRÒ
1	Tdm_counter	Tdm_counter.v	Nguồn thời gian duy nhất. Bộ đếm tự do 5-bit bit_count (0 -> 31 -> 0), xuất tín hiệu chọn kênh trực tiếp channel_select = bit_count[4:3].
2	Tdm_mux	Tdm_mux.v	Bộ ghép kênh tổ hợp 4-to-1. Chọn byte dữ liệu 8-bit của kênh hiện tại để chuyển đổi nối tiếp.
3	Tx_shift_reg	Tx_shift_reg.v	Thanh ghi dịch song song sang nối tiếp (parallel-to-serial, MSB-first). Nạp byte mới khi có xung load, nếu không sẽ dịch trái 1 bit/clock
4	Rx_shift_reg	Rx_shift_reg.v	Thanh ghi dịch nối tiếp sang song song (serial-to-parallel, MSB-first). Dịch liên tục; khi tới byte_boundary sẽ chốt vào rx_data và phát xung byte_done.
5	Tdm_demux	Tdm_demux.v	Bộ tách kênh định tuyến byte hoàn chỉnh vào đúng thanh ghi CHX_OUT, điều phối bởi xung byte_done
6	Tdm_top	Tdm_top.v	Module cấp cao nhất (Top-level). Kết nối 5 module con và chứa lớp keo thời gian (glue logic) tạo tín hiệu điều khiển lệch pha chuẩn xác.

4. TIMING CONTRACT & GLUE LOGIC

Do các thanh ghi dịch tx_shift_reg và rx_shift_reg hoạt động đồng bộ với xung nhịp, dữ liệu nạp chỉ có hiệu lực ở cạnh xung nhịp tiếp theo. Ba thành phần logic keo nhỏ trong tdm_top chịu trách nhiệm dịch chuyển tín hiệu điều khiển thô từ bộ đếm thành các tín hiệu lệch pha chính xác mà không cần sửa đổi 5 sub-module con:

4.1 Tín hiệu mux_select = (bit_count + 1)[4:3] (Logic nhìn trước - Look-Ahead)

- **Mục đích:** Nhìn trước 1 bước thời gian để tx_shift_reg nạp được byte của kênh tiếp theo đúng 1 chu kỳ xung nhịp trước khi bắt đầu quá trình dịch bit ra ngoài.
- **Tầm quan trọng:** Nếu không có cơ chế nhìn trước này, danh sách kênh đầu ra sẽ bị xoay vĩnh viễn 1 vị trí (dữ liệu đầu vào AA/BB/CC/DD sẽ xuất hiện ở đầu ra dưới dạng DD/AA/BB/CC).

4.2 Tín hiệu load và byte_boundary = (bit_count[2:0] == 3'b111)

- **Mục đích:** Kích hoạt xung đúng vào bit cuối cùng của byte hiện tại (3'b111) thay vì bit đầu tiên của byte tiếp theo, giúp việc nạp (TX) và chốt dữ liệu (RX) diễn ra hoàn toàn chính xác theo nhịp.

4.3 Tín hiệu demux_select = channel_select (Trễ 1 xung nhịp)

- **Mục đích:** Làm trễ tín hiệu chọn kênh 1 chu kỳ đồng hồ nhằm giữ cho bộ chọn kênh giải mã đồng bộ với xung byte_done (vốn là một xung được chốt trễ 1 chu kỳ từ rx_shift_reg).

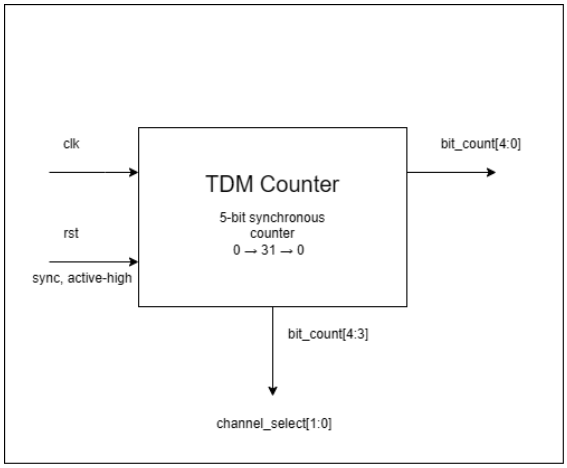
4.4 Logic nạp trước khi Reset (Preload on Reset)

- Trong tx_shift_reg, khi gặp tín hiệu reset, thanh ghi nạp trước giá trị mux_out thay vì xóa về 8'h00. Do bit_count được giữ ở 0 khi reset, mux_select đã trỏ sẵn vào CH0, giúp CH0 trở thành kênh đầu tiên hoàn thành vòng truyền ngay sau khi giải phóng reset.

5. ĐẶC TẢ GIAO DIỆN CÁC MODULE (INTERFACE SPECIFICATIONS)

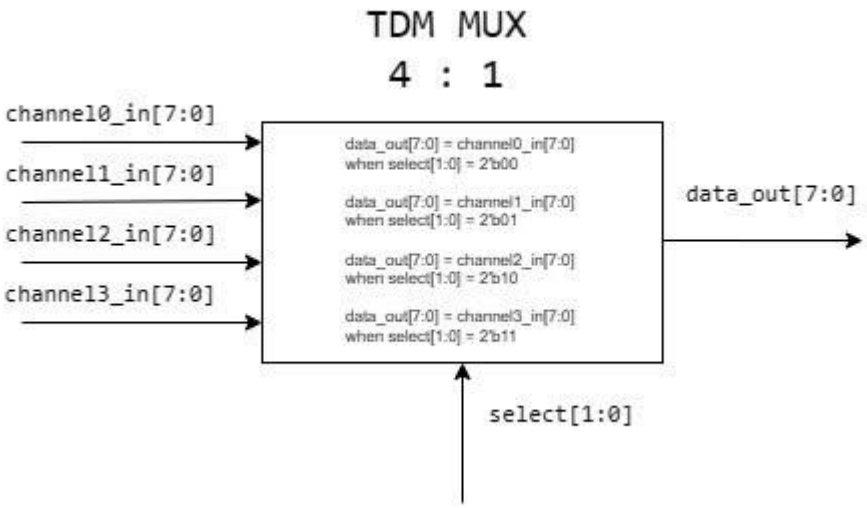
5.1 Module tdm_counter

- **Đầu vào (Inputs):**
  - clk (1-bit): Xung nhịp hệ thống chung.
  - rst (1-bit): Tín hiệu reset đồng bộ, tích cực mức cao (active-high).
- **Đầu ra (Outputs):**
  - bit_count (5-bit): Bộ đếm tự do 0 -> 31 -> 0, nguồn nhịp chính của hệ thống.
  - channel_select (2-bit): Giá trị bit_count[4:3] (dạng thô/chưa chốt).



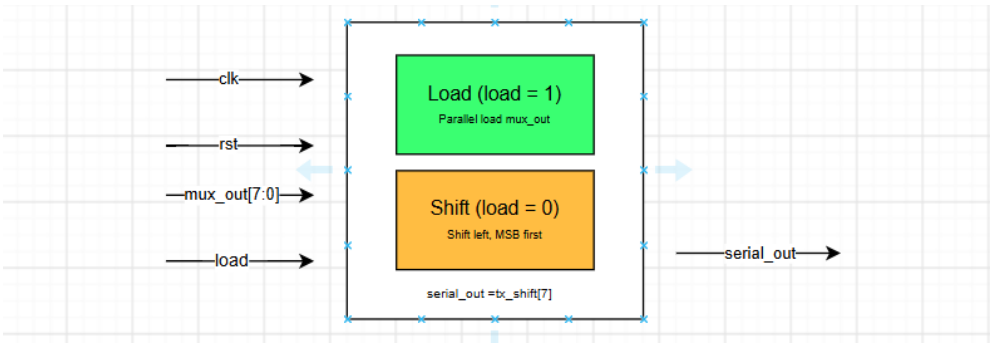
5.2 Module tdm_mux (Multiplexer 4:1)

- **Đầu vào (Inputs):**
  - CH0 .. CH3 (8-bit x 4): 4 kênh đầu vào song song.
  - channel_select (2-bit): Tín hiệu chọn kênh (00 -> CH0, 01 -> CH1, 10 -> CH2, 11 -> CH3).
- **Đầu ra (Outputs):**
  - mux_out (8-bit): Byte dữ liệu của kênh được chọn (xử lý hoàn toàn bằng mạch tổ hợp).



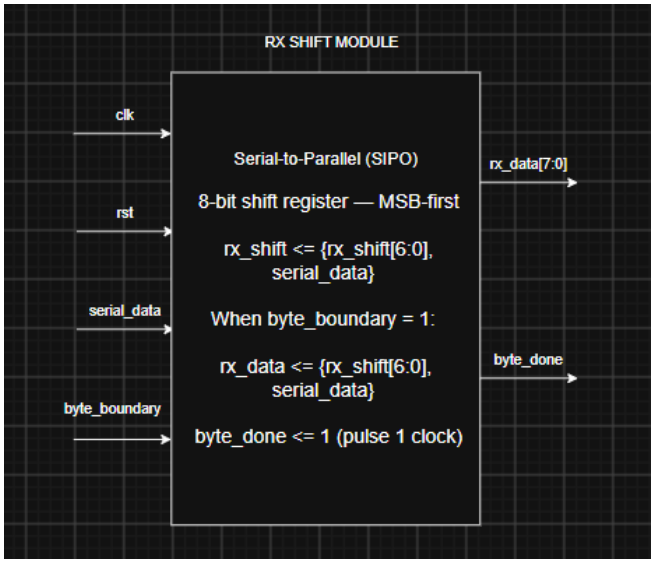
5.3 Module tx_shift_reg (Parallel-to-Serial Shift Register)

- **Đầu vào (Inputs):**
  - clk, rst (1-bit): Clock và Reset đồng bộ tích cực mức cao.
  - mux_out (8-bit): Dữ liệu byte nạp từ tdm_mux.
  - load (1-bit): Xung kích hoạt nạp dữ liệu (1 xung/byte).
- **Đầu ra (Outputs):**
  - serial_out (1-bit): Đầu ra nối tiếp (tx_shift[7], bit MSB hiện tại).
- **Độ ưu tiên logic:** RESET > LOAD > SHIFT.



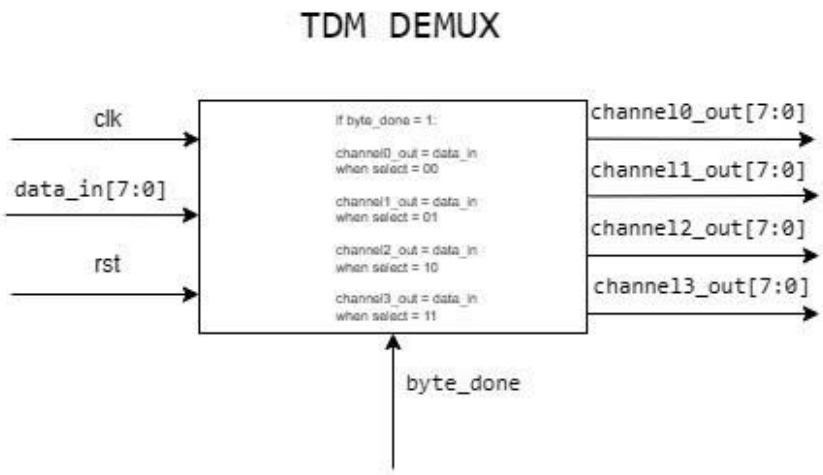
5.4 Module rx_shift_reg (Serial-to-Parallel Shift Register / SIPO)

- **Đầu vào (Inputs):**
  - clk, rst (1-bit): Clock và Reset đồng bộ.
  - serial_data (1-bit): Bit nối tiếp lấy mẫu 1 lần/xung nhịp từ TDM_DATA.
  - byte_boundary (1-bit): Xung báo điểm đầu/cuối byte (vai trò tương tự load ở phía TX).
- **Đầu ra (Outputs):**
  - rx_data (8-bit): Byte khôi phục hoàn chỉnh (MSB-first).
  - byte_done (1-bit): Xung báo hoàn thành byte (xuất hiện trễ 1 chu kỳ sau byte_boundary).



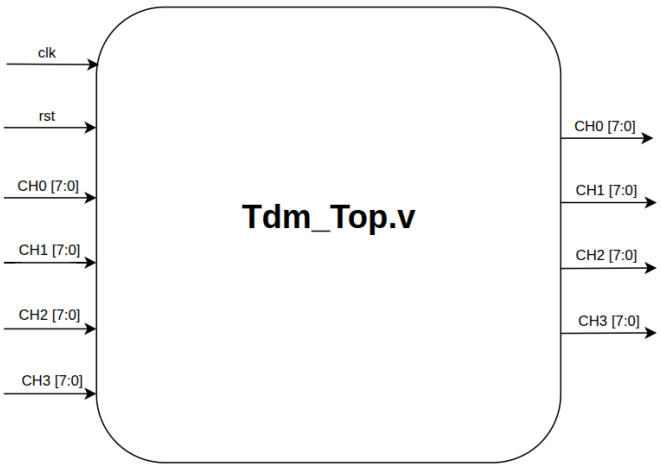
5.5 Module tdm_demux (Demultiplexer / Byte Router)

- **Đầu vào (Inputs):**
  - clk, rst (1-bit): Clock và Reset đồng bộ tích cực mức cao.
  - rx_data (Input, 8-bit): Byte dữ liệu nhận từ rx_shift_reg.
  - channel_select (2-bit): Tín hiệu chọn kênh đầu ra cần cập nhật.
  - byte_done (1-bit): Tín hiệu cho phép ghi (Update gate, ghi khi ở mức 1).
- **Đầu ra (Outputs):**
  - CH0_OUT .. CH3_OUT (8-bit x 4): Các kênh đầu ra khôi phục (giữ nguyên giá trị cũ nếu không được chọn).



5.6 Module tdm_top (Top-Level Integration)

- **Đầu vào (Inputs):**
  - clk, rst (1-bit): Clock và Reset hệ thống.
  - CH0 .. CH3 (8-bit x 4): 4 kênh dữ liệu đầu vào song song.
- **Đầu ra (Outputs):**
  - TDM_DATA (1-bit): Đường truyền dữ liệu nối tiếp duy nhất.
  - CH0_OUT .. CH3_OUT (8-bit x 4): 4 kênh dữ liệu đầu ra đã khôi phục.



6. XÁC THỰC VÀ KIỂM THỬ (VERIFICATION)

6.1 Kiểm thử hệ thống (python scripts/run_tests.py)

- **Kết quả:** Pass **38 / 38** bài kiểm tra trên cả Icarus Verilog (iverilog) và Vivado XSIM.
- **Phạm vi bao phủ (3 lớp kiểm tra độc lập):**
    1. Lớp 1: Kiểm tra truyền nhận trọn gói (round-trip) từng khung bao gồm các trường hợp biên (0x00, 0xFF) và phát sinh reset giữa dòng dữ liệu (mid-stream reset).
    2. Lớp 2: So sánh mức bit toàn bộ 32 bits trên đường truyền TDM_DATA với chuỗi bit MSB-first kỳ vọng (đảm bảo lỗi TX và lỗi RX triệt tiêu lẫn nhau không thể bị che giấu).
    3. Lớp 3: Kiểm thử hồi quy (regression test) với 20 khung ngẫu nhiên liên tiếp (back-to-back).
- **Khả năng tương thích trình mô phỏng (Simulator Portability):** tb_tdm_top giải phóng reset trễ #1 sau cạnh xung nhịp để tránh xung đột thứ tự gán dữ liệu trong cùng một time-step với DUT.

6.2 Đối chiếu với Mô hình chuẩn Python (Python Golden Model vs RTL)

Tệp tb_tdm_top_filebased.v đọc các vector đầu vào từ input/*.hex, kích hoạt tdm_top và ghi kết quả ra tệp. Tệp run_verification.py so sánh từng vector với mô hình Python độc lập:

7. KẾT QUẢ TỔNG HỢP VÀ PHÂN TÍCH VIVADO (SYNTHESIS & TIMING RESULTS)

- **Công cụ tổng hợp:** Xilinx Vivado 2020.2
- **Dòng chip FPGA mục tiêu:** Kintex-7 (xc7k70tfbv676-1)
- **Tần số xung nhịp mục tiêu:** 100 MHz (Ràng buộc chu kỳ 10 ns trong constraints.xdc)

7.1 Báo cáo tiêu thụ tài nguyên phần cứng (Resource Utilization)

Tài nguyên phần cứng	Đã dùng (Used)	Sẵn có (Available)	Tỷ lệ sử dụng (%)
Slice LUTs	**22**	41,000	0.05%
Slice Registers (Flip-Flops)	**63**	82,000	0.08%
Bonded IOB	**67**	300	22.33%
Block RAM (BRAM)	**0**	135	0%
DSP Blocks	**0**	240	0%
BUFGCTRL (Clock buffer)	**1**	32	3.13%

Chi tiết phân rã nguyên thủy (Primitive breakdown):

- 63 FDRE (thiết kế đồng bộ hoàn toàn, không có latch).
- LUT logic: 15 LUT6, 6 LUT3, 3 LUT5, 2 LUT4, 1 LUT2, 1 LUT1.
- I/O primitives: 34 IBUF, 33 OBUF.
- Clock buffer: 1 BUFG.
- Nhận xét: Chân I/O chiếm tỷ lệ lớn nhất (22.33%) do truyền 48-bit đầu vào và 48-bit đầu ra song song cùng clock/reset/serial; thiết kế hoàn toàn không tốn tài nguyên BRAM hay DSP.

7.2 Phân tích thời gian (Design Timing Summary @ 100 MHz / 10 ns)

Loại kiểm tra thời gian (Check)	Slack xấu nhất (Worst Slack)	Endpoint lỗi	Tổng số Endpoint	Trạng thái
Setup (WNS)	**+6.574 ns**	0	104	**Đạt (TIMING MET)**
Hold (WHS)	**+0.159 ns**	0	104	**Đạt (TIMING MET)**
Pulse Width (WPWS)	**+4.600 ns**	0	65	**Đạt (TIMING MET)**

Đánh giá hiệu năng thời gian:

- Tất cả các ràng buộc thời gian do người dùng chỉ định đều đạt chuẩn (TNS = 0.000 ns, THS = 0.000 ns, TPWS = 0.000 ns).
- Từ biên setup slack (+6.574 ns), chu kỳ xung nhịp tối thiểu ước tính 10 - 6.574 = 3.43ns , tương đương với tần số hoạt động tối đa ước tính đạt **khoảng 290 MHz**.

8. PHÂN TÍCH SO SÁNH VÀ ĐÁNH ĐỔI THIẾT KẾ (COMPARISON & TRADE-OFF ANALYSIS)

Trong quá trình thiết kế hệ thống truyền thông số, việc lựa chọn kiến trúc phần cứng và phương pháp kiểm thử đóng vai trò quyết định đến hiệu năng, tài nguyên chip và độ tin cậy. Dưới đây là các phân tích so sánh và đánh đổi (trade-offs) quan trọng của dự án:

8.1 So sánh Kiến trúc: Counter-Driven (Hiện tại) vs. FSM-Driven (Truyền thống)

- **Kiến trúc điều khiển bằng bộ đếm (Counter-driven - Đề tài sử dụng):**

    - *Ưu điểm:* Đường dữ liệu (datapath) hoàn toàn tuần hoàn, loại bỏ hoàn toàn các trạng thái không hợp lệ hoặc nguy cơ kẹt trạng thái (stuck state) thường gặp ở FSM. Tiết kiệm tối đa tài nguyên phần cứng (chỉ tốn 22 LUTs và 63 FFs).
    - *Nhược điểm:* Tính linh hoạt thấp hơn nếu muốn thay đổi giao thức phức tạp giữa dòng mà không reset.
- **Kiến trúc điều khiển bằng Máy trạng thái hữu hạn (FSM-driven):**
    - *Ưu điểm:* Dễ quản lý các giao thức có cơ chế bắt tay (handshake) phức tạp hoặc nhiều chế độ hoạt động khác nhau.
    - *Nhược điểm:* Tốn thêm tài nguyên cho logic chuyển trạng thái, dễ phát sinh lỗi logic (bug) ở các trạng thái biên nếu thiết kế không cẩn thận.

8.2 So sánh Giao tiếp: TDM Nối tiếp 1 đường dây vs. Truyền song song (Parallel Data Bus)

- **Giao tiếp TDM (Time Division Multiplexing - Đề tài sử dụng):**
    - *Ưu điểm:* Tiết kiệm chân I/O trên FPGA cực kỳ hiệu quả (chỉ dùng duy nhất 1 đường dây `TDM_DATA` để truyền toàn bộ 4 kênh thay vì cần tới 32 đường dây riêng biệt). Giảm thiểu diện tích đi dây trên bo mạch (PCB routing).
    - *Nhược điểm:* Phát sinh độ trễ serializing (mất 8 chu kỳ xung nhịp để truyền xong 1 byte) và băng thông tức thời của từng kênh bị chia sẻ theo thời gian.
- **Truyền song song truyền thống:**
    - *Ưu điểm:* Tốc độ truyền dữ liệu tức thời cao, không mất chu kỳ chuyển đổi nối tiếp - song song.
    - *Nhược điểm:* Số lượng chân IOB trên FPGA tăng phi mã, gây quá tải phần cứng khi số lượng kênh mở rộng lớn.

8.3 So sánh Phương pháp Kiểm thử: Python Golden Model vs. Verilog Testbench thuần

- **Mô hình chuẩn Python (Python Golden Model - Đề tài sử dụng):**
    - *Ưu điểm:* Tự động hóa hoàn toàn việc tạo vector ngẫu nhiên số lượng lớn (200+ vectors) và đối chiếu bit-for-bit từng nhịp đồng hồ một cách chính xác tuyệt đối mà không cần viết các câu lệnh `$display` hay `if/else` thủ công phức tạp trong Verilog.
    - *Nhược điểm:* Cần thiết lập môi trường chạy đa ngôn ngữ (Python kết hợp với Icarus Verilog hoặc Vivado XSIM).
- **Testbench Verilog thuần truyền thống:**
    - *Ưu điểm:* Chạy độc lập hoàn toàn trong trình mô phỏng HDL mà không phụ thuộc vào công cụ bên ngoài.
    - *Nhược điểm:* Khó viết các kịch bản kiểm thử ngẫu nhiên phức tạp và việc kiểm tra kết quả thủ công dễ bỏ sót các trường hợp biên (edge cases).

9. HẠN CHẾ VÀ HƯỚNG PHÁT TRIỂN (LIMITATIONS & FUTURE WORK)

9.1 Hạn chế hiện tại (Limitations)

1. **Thứ tự kênh khi thay đổi dữ liệu giữa dòng:** Thứ tự kênh sau khi thay đổi dữ liệu giữa dòng (mid-stream data change) chưa cố định hoàn toàn; có thể khắc phục bằng cách thêm bốn thanh ghi đệm đầu vào chỉ cập nhật khi bit_count quay vòng về 0.

2. **Các trường hợp biên hiếm gặp:** Chưa phủ kín các trường hợp biên như reset liên tiếp (back-to-back resets), xung reset cực ngắn, hay hiện tượng jitter của xung nhịp.

3. **Tham số cứng (Hard-coded):** Số lượng kênh và độ rộng dữ liệu hiện đang cố định, có thể tổng quát hóa bằng các tham số NUM_CHANNELS / DATA_WIDTH kết hợp với vòng lặp generate.

4. **Tính năng kiểm lỗi:** Chưa có cơ hệ phát hiện lỗi hay mã đồng bộ khung (error detection / frame sync marker). Việc bổ sung thêm 1 bit chẵn lẻ (parity bit) cho mỗi byte và một bộ phát hiện mã đánh dấu nhỏ là hoàn toàn khả thi.

5. **Nằm ngoài phạm vi (Out of scope):** Chưa hỗ trợ phát hiện lỗi CRC, khôi phục xung nhịp (clock recovery), hay xử lý băng qua miền xung nhịp (clock-domain crossing) cho các xung nhịp phát/thu tách biệt vật lý.

9.2 Hướng phát triển tương lai (Future Work)

1. **Bổ sung tệp báo cáo thời gian Vivado:** Thêm tệp báo cáo thời gian (.rpt) vào thư mục results/ và ràng buộc độ trễ I/O để các đường dẫn đầu vào và đầu ra cũng được phân tích thời gian đầy đủ.

2. **Cấu hình trên bo mạch mục tiêu:** Lựa chọn một bo mạch FPGA cụ thể, thiết lập tần số xung nhịp thực tế và cấu hình gán chân (pin assignments) trong tệp constraint/constraints.xdc.

3. **Triển khai thanh ghi đệm đầu vào:** Thêm các thanh ghi đệm đầu vào như đã mô tả và mở rộng mô hình chuẩn (golden model) để kiểm chứng chúng.

4. **Phủ kín trường hợp biên:** Bổ sung các testbench để bao phủ các trường hợp biên còn lại (xung reset ngắn, reset liên tiếp).

10. KẾT LUẬN (CONCLUSION)

Dự án thiết kế bộ ghép kênh và tách kênh phân chia theo thời gian (TDM 4-Channel Multiplexer/Demultiplexer) đã hoàn thành xuất sắc các mục tiêu đề ra về mặt kỹ thuật, chức năng và hiệu năng phần cứng:

- **Về mặt kiến trúc và thiết kế:** Hệ thống đã chứng minh tính hiệu quả của phương pháp điều khiển bằng bộ đếm đồng bộ (counter-driven architecture) kết hợp lớp keo logic (glue logic) chuẩn xác, loại bỏ hoàn toàn sự phức tạp của máy trạng thái (FSM) nhưng vẫn đảm bảo luồng dữ liệu tuần hoàn trơn tru. Thiết kế giúp tối ưu hóa số lượng chân I/O đáng kể khi truyền 4 kênh dữ liệu 8-bit qua một đường dây duy nhất (TDM_DATA).
- **Về mặt kiểm thử và xác thực:** Môi trường kiểm thử toàn diện với 38/38 ca kiểm tra hệ thống kết hợp cùng mô hình chuẩn Python (Python Golden Model) chạy 200/200 vector ngẫu nhiên khớp tuyệt đối bit-for-bit đã khẳng định tính đúng đắn, ổn định và độ tin cậy cao của mã nguồn Verilog RTL.
- **Về mặt tổng hợp phần cứng:** Kết quả tổng hợp trên dòng FPGA Kintex-7 thông qua công cụ Vivado 2020.2 cho thấy thiết kế cực kỳ tối ưu về tài nguyên (chỉ chiếm 22 LUTs và 63 FFs, không cần dùng BRAM hay DSP) và đáp ứng vượt mức ràng buộc thời gian ở tần số 100 MHz (với biên độ WNS +6.574ns, tiềm năng hoạt động ổn định ở tần số cao lên tới ~290 MHz)

Đóng góp thành viên (TEAM MEMBERS' CONTRIBUTION)

Class: IC2010		**Team: 3**		**Contribution**
#	**Student ID**	**Student Name**	**Task**	**%**
1	SE205188	CAO HỮU AN	Module TDM_DEMUX + REPORT	16%
2	SE204386	PHẠM MINH TÙNG	Module TDM_MUX + TDM_TOP + TESTBENCH + Kết nối	20%
3	SE205068	MAI THẾ TÂM	Module TDM_Counter	16%
4	SE202014	LÊ THẾ TRƯỜNG	Module TX_SHIFT_REG	16%
5	SE204511	TRẦN MINH TRIẾT	GOLDEN MODEL	16%
6	SE204664	PHAN ANH MINH	Module RX_SHIFT_REG	16%
Total:				**100%**

Tài liệu tham khảo (REFERENCES)

[1] IEEE Standard for Verilog Hardware Description Language

[2] Xilinx Kintex-7 FPGA Data Sheet: Overview

[3] Vivado Design Suite User Guide: Logic Simulation

[4] Vivado Design Suite User Guide: Synthesis

[5] Python HDL Verification Frameworks and Golden Model Methodologies

[6] Advanced Digital Design with the Verilog HDL

[7] RTL Modeling with the IEEE Standard for Verilog

[8] Digital Integrated Circuit Design: From VLSI Architectures to CMOS Fabrication

[9] Time-Division Multiplexing (TDM) Architecture and Applications in Digital Communications

[10] Icarus Verilog (iverilog) Open-Source Simulation Tool Documentation